

Crypto Wars in Secure Messaging: Covert Channels in Signal Despite Leaked Keys

Mohammadamin Rakeei
University of Luxembourg
Esch-sur-Alzette, Luxembourg
amin.rakeei@uni.lu

Rosario Giustolisi
IT University of Copenhagen
Copenhagen, Denmark
rosig@itu.dk

Andy Rupp
University of Luxembourg &
KASTEL Security Research Labs
Esch-sur-Alzette, Luxembourg
andy.rupp@uni.lu

Chuanwei Lin
University of Luxembourg &
Karlsruhe Institute of Technology
Karlsruhe, Germany
chuanwei.lin@kit.edu

Gabriele Lenzini
University of Luxembourg
Esch-sur-Alzette, Luxembourg
gabriele.lenzini@uni.lu

Abstract—End-to-end encryption (E2EE) is the foundation of modern secure messaging, with the Signal protocol as the de facto standard in applications such as Signal, WhatsApp, Facebook Messenger and Google Messages. At the same time, the deployment of E2EE has led to growing pressure from authorities to decrypt user traffic under lawful enforcement. This raises a critical question: if an adversary can routinely decrypt Signal messages (for example via a mandated access or a leaked key), can users still communicate securely and covertly?

We address this question through the lens of anamorphic encryption, which enables hidden communication within seemingly legitimate ciphertexts, even against an adversary who can decrypt them. We design two constructions that embed covert channels into the existing Signal Double Ratchet protocol. Concretely, we show how to embed covert messages (i) into Diffie-Hellman keys used in the asymmetric ratchet, or (ii) into authentication tags produced in the symmetric ratchet. Our techniques are compatible with existing Signal-style deployments and require no changes by the service provider.

We formalize security in threat models that capture adversaries with decryption capabilities granted through lawful-access mechanisms, and prove that the resulting protocol transcripts are indistinguishable from those of standard Signal. We implement our constructions in the official Signal library and Android client, and show that they incur low overhead and are practical in real-world settings. Our results show that covert communication channels can persist even when conventional E2EE guarantees are compromised.

1. Introduction

Secure messaging has become a central part of modern digital communication, especially as users grow more

aware of privacy threats and surveillance. Protocols such as Signal aim to protect users by offering strong end-to-end encryption (E2EE), ensuring that only the sender and receiver can access the contents of a message. Even if the messaging server is compromised, or if an adversary tries to intercept communication, these protocols aim to keep messages private and secure. The Signal protocol stands out as the most widely adopted secure messaging protocol today. It is used not only in the Signal app itself but also as the foundation for messaging in WhatsApp, Facebook Messenger, Google Message and other major platforms. Signal’s design combines two main components: the X3DH key agreement protocol [PM16b] and the Double Ratchet algorithm [PM16a]. Together, they allow for asynchronous messaging with forward secrecy, post-compromise security, and cryptographic deniability. At the heart of Signal’s security is the Double Ratchet, which updates keys after each message to protect future and past communication, even if a device is temporarily compromised. These properties have been the subject of extensive security analysis and are widely considered a strong defense against powerful attackers [KBB17], [CJSV22], [BFG⁺22], [CGCD⁺20], [ACD19], [VGIK20], [CRT24].

1.1. Motivation

E2EE messaging is increasingly targeted by political and institutional efforts to introduce surveillance, typically by weakening or bypassing E2EE, often justified as lawful access or national security. Among the various use cases, the most notable are the efforts by the EU [Eur25], UK [BBC23], Sweden [dig25], and more recently the Australian government [cyb25], asking Signal to undermine its E2EE to allow the government to decrypt all encrypted messages.

As a concrete illustration of this pressure, the EU’s so-called *chat control* policy, based on client-side scanning, moves inspection onto users’ devices [EU 25]. This approach has been challenged by cryptography and security researchers who argue that on-device scanning weakens E2EE, erodes privacy, enlarges the attack surface, and normalizes surveillance with a high risk of function creep ¹.

We consider a general model that, when activated, enables the authorities to decrypt messages. How this capability is achieved is left open. It may arise through client-side scanning hooks, through a platform-run key escrow where user keys are stored encrypted under a law enforcement public key with access granted only on the basis of a warrant, or through other policy-driven mechanisms under active debate worldwide. Our approach is general and does not depend on how surveillance is realized. We ask whether covert, indistinguishable communication can persist within the Double Ratchet under this key disclosure model.

This paper addresses a fundamental question:

If Signal’s E2EE is compromised under authoritarian control, can users still communicate securely and undetectably?

Our goal is to design mechanisms that enable hidden communication between sender and receiver, while preserving the normal behavior of the protocol, even when Signal is no longer trustworthy in the traditional sense. We propose two constructions that preserve secure and undetectable communication under key disclosure. In both cases, we assume the message decryption key, possibly along with other user secrets, is revealed, so the confidentiality of ordinary Signal messages is broken. We achieve this goal with *anamorphic encryption* [PPY22], a recent approach that allows the sender and receiver to have covert communication by hiding sensitive messages inside other normal-looking ciphertexts, even when the adversary can decrypt messages. Our proposed constructions produce cryptographic elements that are indistinguishable from normal Signal outputs, making hidden communication possible without changing the structure or behavior of the protocol.

1.2. Contributions

We design and integrate two novel anamorphic channels into the Signal Double Ratchet protocol. The first construction embeds hidden messages into the Diffie-Hellman (DH) key pairs generated in the asymmetric ratchet; the second embeds hidden data into MAC authentication tags in the symmetric ratchet. These constructions are compatible with existing Signal implementations, require no structural changes, and rely solely on standard cryptographic assumptions. To the best of our knowledge, no previously proposed anamorphic construction can be applied to the Signal Double Ratchet protocol or to an existing E2EE messaging application.

Each construction targets a key-exposure scenario. For the DH-based channel, we support either a symmetric double key shared by the endpoints or an asymmetric double public key published by the endpoints, covering cases where the server-side leakage includes Double Ratchet state but excludes the hidden double secret. For the MAC-based channel, we address settings in which either the root key or the private DH ratchet key is leaked, but not both, so that the derived MAC keys remain secret.

We formalize security via indistinguishability of anamorphic executions from standard Signal Double Ratchet transcripts, which will be referred as anamorphic security. And we extend the base constructions with transcript-binding upgrade such that the construction resist attacks in an IND-CCA2-style sense.

We provide concrete instantiations and optimizations. The DH-based channel is instantiated over X25519 using Edwards–Montgomery mappings, together with a lookup-table optimisation that replaces an exhaustive search over messages with one group operation and a table lookup, making a 32-bit covert payload per DH step practical. The MAC-based channel supports payloads up to the MAC size; in our implementation this is 64 bits.

We implement both channels in a Signal-compatible Double Ratchet library and Android client, evaluate their performance, and observe low overhead. The designs require no server-side changes, and remain compatible with existing Signal-style deployments.

These results indicate that covert messaging remains feasible even when traditional E2EE guarantees are compromised, and they extend the scope of anamorphic encryption to real-world secure messaging systems.

2. Background

This work primarily focuses on the two main components of the Double Ratchet protocol, namely, DH and symmetric ratchets. Therefore, we focus on the cryptographic primitives used in these two components, and refer the reader to [PM16a] for additional details.

2.1. Cryptographic functions

The Double Ratchet protocol is primarily built upon ECDH key exchange and the HKDF key derivation function [Kra10]. In the following, we assume familiarity with standard cryptographic notions and outline the relevant cryptographic functions, specifying for each its interface, implementation, and key features.

- $\text{Keygen}(\lambda) \rightarrow (\text{sk}, \text{pk})$: Given a security parameter λ , the key generation algorithm outputs a key pair (sk, pk) . In the DH ratchet we recall later, it generates an ECDH key pair using either the X25519 or X448 function, denoted as $(\text{sk}, \text{pk} := \text{sk} \cdot G)$, where G is the base point of the elliptic curve, $\text{sk} \in \mathbb{Z}_p$ is a randomly chosen private key, and p is the prime order of the subgroup, denoted as $\mathbb{G}$

1. Security Scientists Open Letter, accessed Nov 2025

with a generator G . The default curve in the Signal protocol is Curve25519. We write $a \cdot G$ as a shorthand notation for G^a , so that $(a \pm b) \cdot G$ represents $G^{a \pm b}$.

- $\text{DH}(\text{sk}_a, \text{pk}_b) \rightarrow \text{dh}_{\text{out}}$: Given A's secret key sk_a and B's public key pk_b , the DH function computes DH shared secret $\text{dh}_{\text{out}} := \text{sk}_a \cdot \text{pk}_b$. Note that $\text{DH}(\text{sk}_a, \text{pk}_b) = \text{DH}(\text{sk}_b, \text{pk}_a)$.
- $\text{PRF}(\text{prf}_{\text{key}}, x) \rightarrow y$: Given a key prf_{key} and an input x , the pseudorandom function (PRF) outputs an element y . It is required that the output be computationally indistinguishable from a value from a truly random function mapping from the same domain to the same codomain.
- $\text{KDF}(\text{kdf}_{\text{key}}, x) \rightarrow y$: Given a key kdf_{key} and an input x , the key derivation function outputs an element y . It is required that the output be computationally indistinguishable from a value sampled uniformly at random from the same range. Moreover, the KDF output can be split into several parts while the knowledge of the partial output should not leak information on the other generated part.

The KDFs for the double ratchet protocol are HMAC-based KDF, consisting of two components: The first extract function takes as input the salt and the input key material, producing a PRF key. Then, the expand function takes the PRF key (along with the implicit auxiliary information and the desired output length) as input, and outputs the derived key. We specify each KDF involved in the ratchet as follows.

- $\text{KDF}_{\text{rk}}(\text{rk}, \text{dh}_{\text{out}}) \rightarrow (\text{rk}_{\text{new}}, \text{ck})$: Given an old root key rk as salt and the shared secret dh_{out} as input key material, the HMAC-based key derivation function (HKDF) generates a new root key rk_{new} and a new chain key ck .
- $\text{KDF}_{\text{ck}}(\text{ck}) \rightarrow (\text{ck}_{\text{new}}, \text{mk})$: Given the old chain key ck , the HKDF generates a new chain key ck_{new} and a message key mk . In this function, a constant value is used as key salt.
- $\text{Enc}(\text{mk}, \text{pt}, \text{ad}) \rightarrow (\text{aes}_{\text{key}}, \text{mac}_{\text{key}}, \text{IV}, \text{ct}, \text{mac})$: Given a message key mk , a plaintext pt and the associated data ad , the encryption algorithm, implemented as Authenticated Encryption with Associated Data (AEAD) in the protocol, outputs an AES key aes_{key} , a MAC key mac_{key} , the initialization vector IV , a ciphertext ct and a MAC mac . It proceeds in the following three steps, combining CBC and HMAC to ensure misuse resistance in cases where a key is inadvertently reused:
 - 1) $\text{KDF}_{\text{mk}}(\text{mk}) \rightarrow (\text{aes}_{\text{key}}, \text{mac}_{\text{key}}, \text{IV})$: Given mk as input key material, the HKDF outputs aes_{key} , mac_{key} and IV . The HKDF salt is a zero-filled sequence that matches the output length of the hash function;
 - 2) $\text{AES}_{\text{CBC}}(\text{aes}_{\text{key}}, \text{IV}, \text{pt}) \rightarrow \text{ct}$: Given aes_{key} , IV and pt , the encryption function, AES-256 in CBC mode with PKCS#7 padding, outputs a ciphertext

ct ;

- 3) $\text{MAC}(\text{mac}_{\text{key}}, \text{ct}, \text{ad}) \rightarrow \text{mac}$: Given mac_{key} , ct and ad , the MAC function returns a mac .

2.2. The Double Ratchet protocol

Many messaging applications use the Signal's Double Ratchet protocol to generate message keys in support of E2EE. This protocol ensures secure messaging by constantly updating encryption keys. It provides both forward secrecy and post-compromise secrecy. In a nutshell, every time a message is sent, a new key is generated, limiting the exposure of past or future messages if a key is compromised. The protocol bootstraps from a shared secret that sender and receiver have previously agreed based on a secure key agreement protocol. In Signal, this protocol is X3DH [PM16b]. The Double Ratchet protocol consists of the symmetric ratchet and the DH ratchet.

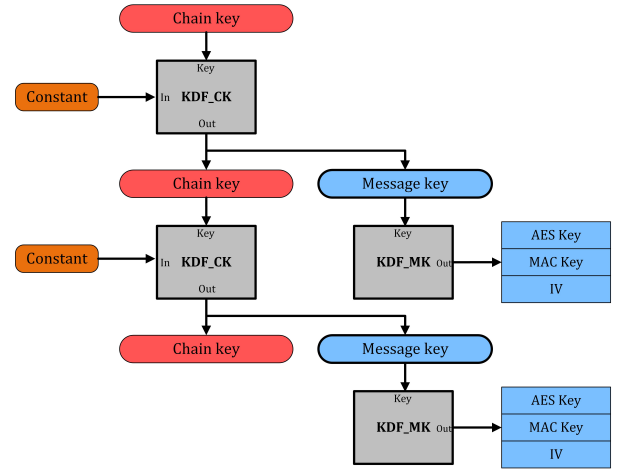


Figure 1: The symmetric ratchet

Symmetric ratchet. The goal of the symmetric ratchet is to generate a new message key for each message sent or received. It involves the use of the functions KDF_{ck} and Enc . The symmetric ratchet initiates with an old chain key, and derives a new chain key and a message key as shown in Figure 1.

DH ratchet. The goal of the DH ratchet is to provide forward secrecy and post-compromise security against an attacker who gains access to a chain key. In the DH ratchet, each party generates a DH key pair, which serves as their current ratchet key pair (sk, pk) . Every message is sent with a header that includes the sender's current public key pk . When one party receives a new public key from the other, they execute DH, replacing their own key pair with a fresh one. This update triggers KDF_{ck} to generate a new chain key, ensuring continuous key update for enhanced security. The DH ratchet is illustrated in Figure 2.

In short, the general rules to run either ratchet can be summarised as follows:

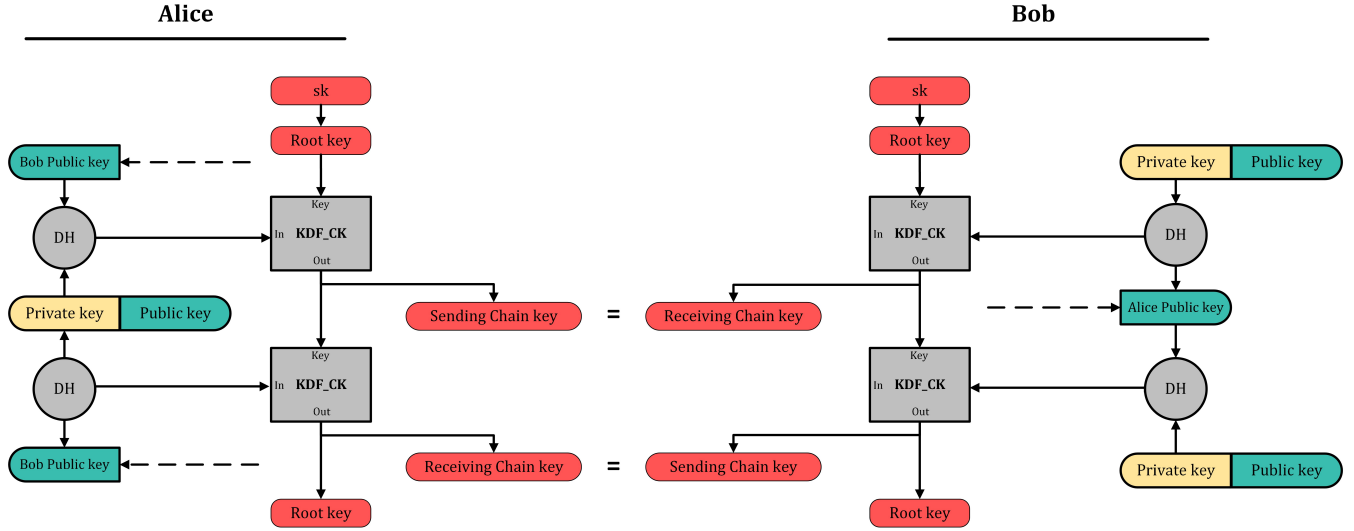


Figure 2: The DH ratchet

- On sending or receiving a new message, a symmetric ratchet step updates the sending or receiving chain to generate a new message key.
- On receiving a new ratchet public key, a DH ratchet step replaces the chain keys before executing the symmetric ratchet.

Combining DH ratchet and symmetric ratchet forms the Double Ratchet protocol as depicted in Figure 3, which illustrates an example of a Double Ratchet scenario from Alice’s point of view. In this flow, Alice receives an initial ratchet public key from Bob, then performs a DH ratchet step to obtain her first sending chain key. She then sends two messages within this chain. Subsequently, Bob replies with a new message, including a new ratchet public key. Alice generates a new receiving chain key and derives the corresponding message keys for the messages received from Bob. The final part of this communication involves Alice sending a message along with a new ratchet public key. Note that Bob performs the same operations on his side, generating the corresponding message keys. Since only Alice and Bob share the initial root key generated via X3DH, no one else can compute the sending and receiving keys, and consequently, the message keys. This shared secret also ensures security against man-in-the-middle attacks by the server, which only relays the communication.

Figure 2 also illustrates that compromising a sending or receiving chain key does not affect the security of future keys, as KDF_{ck} takes in the output of dh_{out} , which is known only to Alice and Bob.

2.3. Anamorphic encryption

Anamorphic encryption [PPY22] protects communication even when a dictator restricts the permitted cryptographic primitives and holds the communication secret key.

It augments the standard encryption mode with an anamorphic mode: a sender can use a *double key* to embed a hidden message that only the intended receiver can recover with the same key pair. To the dictator, the security of an anamorphic encryption scheme guarantees that the keys and the ciphertexts in the standard and anamorphic modes are indistinguishable.

An anamorphic encryption scheme linked to a standard encryption scheme of (Gen, Enc, Dec) consists of three algorithms (aGen, aEnc, aDec). In an anamorphic mode, the aGen algorithm returns an anamorphic public key apk, an anamorphic secret key ask and a double key dk. The aEnc algorithm encrypts a normal message msg together with an anamorphic message amsg with the double key dk into a ciphertext, which can only be decrypted by the aDec algorithm with the double key.

For the Double Ratchet, we design the three anamorphic components in a similar manner. Specifically, there is a double key generation algorithm aGen that samples a double key (pair). Using the double key (pair), the anamorphic message can be encrypted and decrypted. We omit the explicit invocation of double key generation algorithm and assume that, when necessary, the double key pair is generated separately.

3. System Design

The essential properties of E2EE are confidentiality, integrity, authentication, and forward secrecy [KCKG24]. Optional features include availability, loss resilience, deniability, and post-compromise security. As the main goal of surveillance is to monitor users’ messages, in all our threat models we focus on the loss of confidentiality of Signal messages. In the Signal Double Ratchet this is captured by leakage of the per-message encryption key (AES key) to the server.

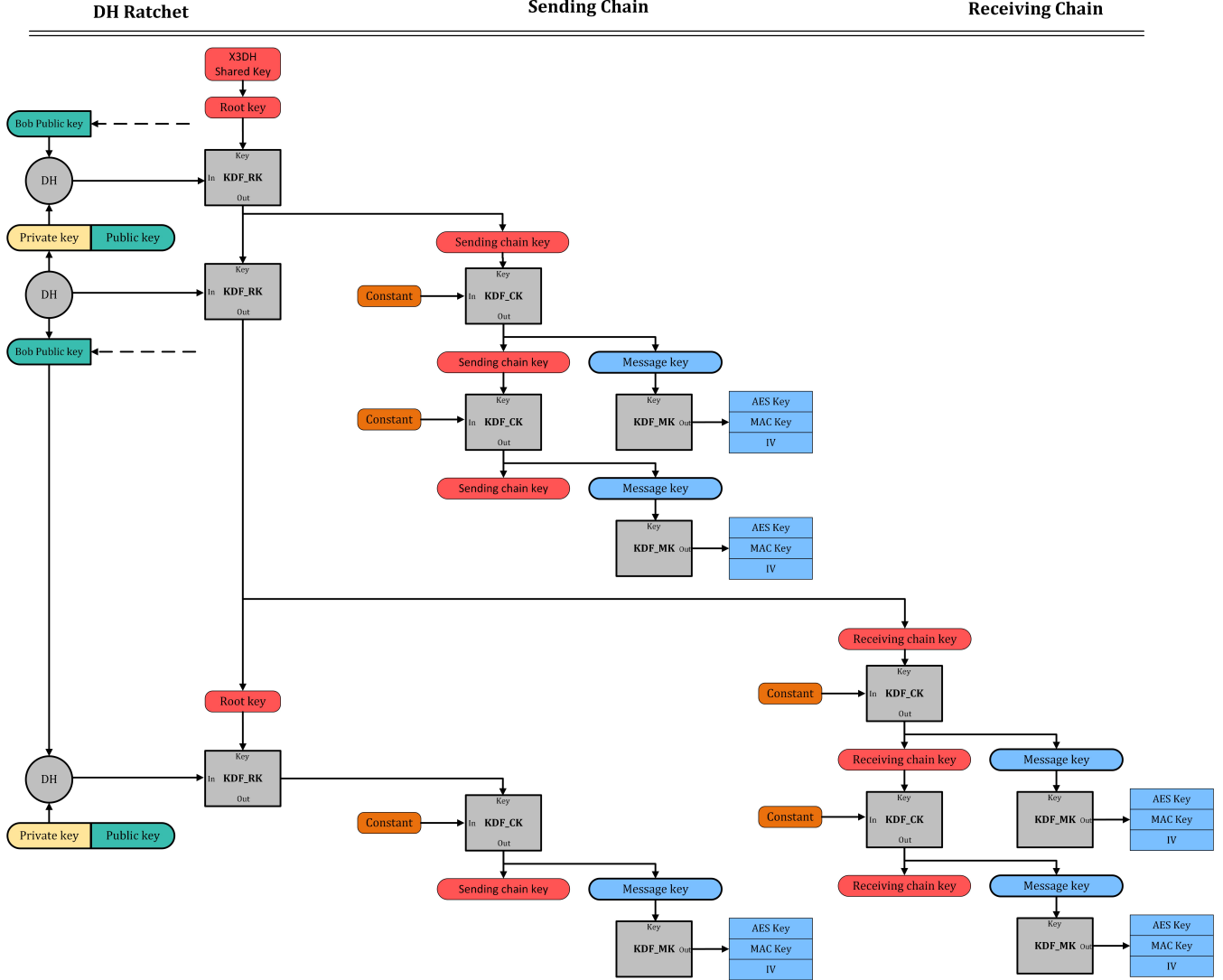


Figure 3: An instance of the Double Ratchet protocol for Alice

TABLE 1: Anamorphic constructions under various threat models. $\diamond$ allows either the Root key or the DH private key to be leaked.

Construction	Leak					No PSK required
	Root key	DH private key	Chain/Message key	MAC key	AES key	
Sym-DH	✓	✓	✓	✓	✓	×
Asym-DH	✓	✓	✓	✓	✓	✓
Sym-MAC	$\diamond$	$\diamond$	×	×	✓	✓

Scope. We consider key-disclosure mechanisms that enable decryption without changing the protocol’s wire format or state machine. For example, apart from the surveillance mechanisms such as key escrow mentioned earlier, one may also consider introducing a backdoor that reveals only the decryption keys. Such designs might modify other parts of the Signal Double Ratchet and are out of scope, as their broader security impact requires separate analysis.

Adversary and setting. We model the Signal server as

corrupted, giving the adversary full monitoring capability over all users’ communications. We discuss our threat models considering both passive adversaries who only monitor the protocol and active ones who can also interact with the protocol.

Key-leakage scenarios. We study disclosures of the Signal Double Ratchet state to the server. Revealing some keys may enable derivation of others; for instance, exposing both the root key and the private DH ratchet key lets the adversary

reconstruct chain and message keys. Throughout, we assume aes_{key} is leaked so that the server can always decrypt plaintexts. Beyond this baseline, we consider scenarios in which the DH ratchet state, the symmetric ratchet state, or both are exposed, as summarized in Table 1. Note that in this table, the DH private key refers to the key securely generated during the DH ratcheting process. We omit the double key generation process and assume that, depending on the construction, the double keys may or may not need to be pre-shared through a secure channel.

System overview. We present two classes of constructions that capture three scenarios, remain compatible with existing Signal deployments and require no server changes:

- 1) a DH-based channel with a symmetric double key shared by the endpoints;
- 2) a DH-based channel with an asymmetric double public key published by the receiver;
- 3) a symmetric-ratchet channel that embeds payloads into MAC tags.

Security goals. For our constructions, we define and prove their *anamorphic security* ensuring that an adversary cannot distinguish the anamorphic mode from the real mode, i.e., it cannot detect whether the observed data (e.g., the public and secret keys or the MACs) contain an anamorphic message or not. Anamorphic security implies IND-CPA security, which one could also define for these settings, since if it is not possible to detect the existence of a chosen anamorphic message (anamorphic security), it is also not possible to decide which of two chosen messages is embedded (IND-CPA). Moreover, we also present a transcript-binding upgrade that attains a stronger, CCA-style guarantee.

4. Covert Channel in DH Ratchet

To embed covert messages into the DH public keys used in the DH ratchet, we construct an anamorphic channel with a hidden double key. For now, we assume that all keys other than this double key may be leaked. Later in this section, we explain how this assumption is instantiated in different scenarios.

4.1. Base Construction

Recall that in the DH ratchet, fresh DH key pairs are generated regularly. We show how to create *fake* DH key pairs that embed covert messages while remaining indistinguishable from honest ones, and we present two algorithms. The first generates the fake DH key pairs and the second extracts the hidden messages.

Here, we provide an abstract description over a group G . A standard DH key generation algorithm samples $\text{sk} \leftarrow \mathbb{Z}_p$ and outputs $\text{pk} = \text{sk} \cdot G \in \mathbb{G}$, where G is the base point of the elliptic curve. We write $a \cdot G$ as a shorthand notation for G^a , so that $(a \pm b) \cdot G$ represents $G^{a \pm b}$.

Algorithm 1 generates $\ell + 1$ key pairs that embed an anamorphic message vector $\overrightarrow{\text{msg}} \in \mathcal{M}^\ell$ using the double

Algorithm 1 fakeKeygen(dk, $\overrightarrow{\text{msg}}$)

```

1: parse  $\overrightarrow{\text{msg}} = \{\text{msg}_i\}_{i=1}^\ell$ 
2:  $(\text{sk}_1, \text{pk}_1) \leftarrow \text{Keygen}(\lambda)$ 
3: for  $i \in [2, \ell + 1]$  do
4:    $\text{sk}_i = \text{PRF}(\text{dk}, \text{pk}_{i-1}) + \text{msg}_{i-1} \bmod p$ 
5:    $\text{pk}_i = \text{sk}_i \cdot G$ 
6: end for
7: return  $\{(\text{sk}_i, \text{pk}_i)\}_{i=1}^{\ell+1}$ 

```

key $\text{dk} \in \{0, 1\}^\lambda$. Here, the pseudorandom function PRF with key space $\{0, 1\}^\lambda$ maps elements from $\mathbb{G}$ into $\mathbb{Z}_p$.

Algorithm 2 recovers the anamorphic message vector $\overrightarrow{\text{msg}} \in \mathcal{M}^\ell$ using the double key dk from PK consisting of $\ell + 1$ public keys pk_i .

Algorithm 2 aDec(dk, PK)

```

1: parse  $\text{PK} = (\text{pk}_i)_{i \in [1, \ell+1]}$ 
2: for  $i \in [2, \ell + 1]$  do
3:    $\text{msg}_{i-1} = \text{dlog}_G(\text{pk}_i - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G)$ 
4: end for
5: return  $\{\text{msg}_i\}_{i=1}^\ell$ 

```

Since it requires performing modular exponentiations to recover anamorphic messages, the anamorphic message space $\mathcal{M}$ should be small. As we show later, it is feasible to choose it up to 16 bits. Note that this and all other constructions ensure robustness: decrypting an anamorphic ciphertext under the correct double key yields a valid anamorphic message, and otherwise returns an explicit abort signal, i.e., when the discrete logarithm computation fails to produce a valid anamorphic message.

To integrate this into the full Double Ratchet protocol, Alice and Bob perform the following steps to transmit ℓ anamorphic messages within the DH ratchet using $\ell + 1$ DH keys. Without loss of generality, we present our construction in the setting where Alice sends covert messages to Bob, and assume for now that Alice and Bob share the same private double key dk .

- 1) Alice runs a fake key generation algorithm $\text{fakeKeygen}(\text{dk}, \overrightarrow{\text{msg}})$ as described in Algorithm 1 with her private double key dk .
- 2) Alice sends $\ell + 1$ Signal messages to Bob. Before sending each message, she waits for a successful delivery notification ².
- 3) Bob collects the last $\ell + 1$ ratchet public keys $\text{PK} = \{\text{pk}_1, \dots, \text{pk}_{\ell+1}\}$ sent by Alice and runs $\text{aDec}(\text{dk}, \text{PK})$ to retrieve $\overrightarrow{\text{msg}}$ described in Algorithm 2.

We now argue correctness and security of the construction. Intuitively, correctness requires that, given the correct double key dk , the anamorphic messages should be recovered from the anamorphic ciphertexts, that is, from the fake DH public keys produced by our algorithms. Anamorphic

2. This is necessary to avoid raising suspicion from the server. If Alice sends all $\ell + 1$ messages at once, it may appear suspicious that she is not using the same sending chain key for some of them. When Bob sends a Signal message, including a successful delivery notification, he also includes a new DH ratchet public key

security requires that it is infeasible to detect the presence of the covert channels: an adversary should not be able to distinguish honestly generated fresh key pairs and the key pairs that embed anamorphic messages.

Correctness. Each anamorphic message amsg_i is correctly recovered since for all $i \in [2, \ell + 1]$, it holds that

$$\begin{aligned} \text{amsg}_{i-1} &= \text{dlog}_G(\text{pk}_i - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G) \\ &= \text{dlog}_G(\text{sk}_i \cdot G - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G) \\ &= \text{dlog}_G((\text{PRF}(\text{dk}, \text{pk}_{i-1}) + \text{amsg}_{i-1}) \cdot G \\ &\quad - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G) \\ &= \text{dlog}_G(\text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G + \text{amsg}_{i-1} \cdot G \\ &\quad - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G) \\ &= \text{dlog}_G(\text{amsg}_{i-1} \cdot G). \end{aligned}$$

Anamorphic Security. The construction achieves anamorphic security if for every probabilistic polynomial-time (PPT) adversary $\mathcal{D}$ it holds that

$$\left| \Pr[\text{GReal}_{\text{Kgen}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GAnam}_{\text{fkKgen}}^{\mathcal{D}}(\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

where the security games are defined as follows:

$\begin{aligned} &\text{GReal}_{\text{Kgen}}^{\mathcal{D}}(\lambda) : \\ &1) \text{ dk} \leftarrow \text{aGen}(\lambda) \\ &2) \text{ return } \mathcal{D}^{\mathcal{O}_{\text{Kgen}}(\cdot)}(\lambda) \text{ where} \\ &\quad \mathcal{O}_{\text{Kgen}}(\overrightarrow{\text{amsg}}) = \{\text{Keygen}(\lambda)\}_{i=1}^{\ell+1} \end{aligned}$ $\begin{aligned} &\text{GAnam}_{\text{fkKgen}}^{\mathcal{D}}(\lambda) : \\ &1) \text{ dk} \leftarrow \text{aGen}(\lambda) \\ &2) \text{ return } \mathcal{D}^{\mathcal{O}_{\text{fkKgen}}(\cdot)}(\lambda) \text{ where} \\ &\quad \mathcal{O}_{\text{fkKgen}}(\overrightarrow{\text{amsg}}) = \text{fakeKeygen}(\text{dk}, \overrightarrow{\text{amsg}}, \ell) \end{aligned}$
--

In the games defined for this construction, we explicitly include the generation of the double key dk , which was omitted in the original construction. This explicit step ensures that in both games, the double key dk in both games is of the same distribution. The value dk is consumed only by the anamorphic oracle and is not revealed to the distinguisher $\mathcal{D}$; the real oracle ignores it. Moreover, in the game here, the $\mathcal{D}$ does not require access to other keys, as they do not influence the generation of DH ratchet key pairs.

Theorem 1. *The construction present in Section 4.1 satisfies anamorphic security if PRF used in Algorithm 1 is a secure PRF, as introduced in Section 2.1.*

Proof. In the following sequence of hybrid games, let $\mathcal{D}$ be a PPT adversary and Q its total number of oracle queries.

Game 0: This is the anamorphic game $\text{GAnam}_{\text{fkKgen}}^{\mathcal{D}}(\lambda)$ in the security definition, where the oracle generates $(\text{pk}_i, \text{sk}_i)$ as described in Algorithm 1, $\text{sk}_i = \text{PRF}(\text{dk}, \text{pk}_{i-1}) + \text{amsg}_{i-1}$, $\text{pk}_i = \text{sk}_i \cdot G$.

Game 1: In the second game, we replace all applications of $\text{PRF}(\text{dk}, \cdot)$ by the application of a uniformly chosen function $\mathcal{F} : \mathbb{G} \rightarrow \mathbb{Z}_p$.

Game 2: In the third game, we define $\mathcal{F}$ iteratively by lazy sampling: each time $\mathcal{F}$ should be applied, we check if we

already established a mapping for the given input a , i.e., $a \mapsto z$ is contained in some set $\mathcal{M}_{\mathcal{F}}$, and if this is the case z is the output. Otherwise, some fresh $z \leftarrow \mathbb{Z}_p$ is sampled uniformly and the mapping $a \mapsto z$ is added to $\mathcal{M}_{\mathcal{F}}$.

Game (3, j) for $j \in [0, \ell Q]$: In this series of games, we slightly modify Game 2 until we end up with Game 4, by modifying the way each of the ℓQ applications of $\mathcal{F}$ is computed. We define Game (3, 0) to be identical to Game 2. In Game (3, j), the challenger handles the first j applications of $\mathcal{F}$ as follows: Instead of checking if the a mapping for the corresponding input is already defined in $\mathcal{M}_{\mathcal{F}}$, it just samples a uniform output and adds the corresponding mapping to $\mathcal{M}_{\mathcal{F}}$. Note that in Game (3, ℓQ) all applications of $\mathcal{F}$ are replaced by uniformly sampling elements from $\mathbb{Z}_p$.

Game 4: This is the real game $\text{GReal}_{\text{Kgen}}^{\mathcal{D}}(\lambda)$ in the anamorphic security definition, where the oracle generates $(\text{pk}_i, \text{sk}_i)$ by choosing $\text{sk}_i \leftarrow \mathbb{Z}_p$ uniformly at random and setting $\text{pk}_i = \text{sk}_i \cdot G$.

Game 0 and Game 1 are computationally indistinguishable, as a distinguisher for them could be turned in to one for the pseudo-randomness property of PRF. The games Game 1 and Game 2 are perfectly indistinguishable as Game 2 just uses an equivalent way to define $\mathcal{F}$. Game (3, 0) coincides with Game 2 by definition. The difference between games Game (3, $j - 1$) and Game (3, j) consists in how the output of the j -th application of $\mathcal{F}$ is computed. In Game (3, $j - 1$) we check whether the input pk_{j-1} occurred before in $\mathcal{M}_{\mathcal{F}}$, while we do not perform this check in Game (3, j). Observing that all pk_i for $i \leq j - 1$ have been chosen uniformly at random from $\mathbb{Z}_p$, the probability that $\text{pk}_i \mapsto z$, for some z , occurs in $\mathcal{M}_{\mathcal{F}}$ is upper bounded by $\frac{j-1}{p}$. Thus, in total the probability that games Game (3, 0) and Game (3, ℓQ) differ is upper bounded by $\frac{(\ell Q)^2}{p}$, which is negligible. Finally, note that Game (3, ℓQ) and Game 4 are identical by observing that $\text{sk}_i = z_i + \text{amsg}_{i-1}$ for $z_i \leftarrow \mathbb{Z}_p$ yields a uniformly random sk_i in Game (3, ℓQ). $\square$

4.2. Stronger Security Guarantee – IND-CCA

We now strength our security guarantee. Analogous to IND-CCA2 security for an encryption scheme, we require that security holds even against an adversary that obtains ciphertexts generated in anamorphic mode and actively interacts with the protocol. In particular, such an adversary must neither be able to distinguish anamorphic executions from honest ones nor learn any information about the embedded anamorphic messages.

Intuitively, the updated fake key generation algorithm binds the covert encoding to the current transcript and adds an extra key pair used as an integrity check. The decryption algorithm is modified accordingly to verify this binding before extracting the anamorphic message. This yields active security at the cost of only one additional key pair.

We update the fake key generation algorithm (Algorithm 1) and the anamorphic message decryption algorithm (Algorithm 2) to Algorithm 3 and Algorithm 4, respectively.

We add an auxiliary PRF* (same key space) that maps the transcript to the secret key, binding keys to the transcript.

Algorithm 3 fakeKeygen(dk, $\overrightarrow{\text{amsg}}$)

```

1: parse  $\overrightarrow{\text{amsg}} = \{\text{amsg}_i\}_{i=1}^\ell$ 
2:  $(\text{sk}_1, \text{pk}_1) \leftarrow \text{Keygen}(\lambda)$ 
3: for  $i \in [2, \ell + 1]$  do
4:    $\text{sk}_i = \text{PRF}(\text{dk}, \text{pk}_{i-1}) + \text{amsg}_{i-1} \bmod p$ 
5:    $\text{pk}_i = \text{sk}_i \cdot G$ 
6: end for
7:  $T = \text{pk}_1 \parallel \text{pk}_2 \parallel \dots \parallel \text{pk}_{\ell+1}$ 
8:  $\text{sk}_{\ell+2} = \text{PRF}^*(\text{dk}, T)$ 
9:  $\text{pk}_{\ell+2} = \text{sk}_{\ell+2} \cdot G$ 
10: return  $\{(\text{sk}_i, \text{pk}_i)\}_{i=1}^{\ell+2}$ 

```

Algorithm 4 aDec(dk, PK)

```

1: parse  $\text{PK} = (\text{pk}_i)_{i \in [1, \ell+2]}$ 
2:  $T = \text{pk}_1 \parallel \text{pk}_2 \parallel \dots \parallel \text{pk}_{\ell+1}$ 
3: if  $\text{pk}_{\ell+2} \neq \text{PRF}^*(\text{dk}, T) \cdot G$  then
4:   return  $\perp$ 
5: end if
6: for  $i \in [2, \ell + 1]$  do
7:    $\text{amsg}_{i-1} = \text{dlog}_G(\text{pk}_i - \text{PRF}(\text{dk}, \text{pk}_{i-1}) \cdot G)$ 
8: end for
9: return  $\overrightarrow{\text{amsg}}$ 

```

We defer the security proof to Appendix A

4.3. Instantiation of Double Key dk

So far, our covert channel in the DH keys of the DH ratchet assumes that Alice and Bob share a double key dk. We now show how to instantiate this assumption under different threat models.

Case 1: Symmetric using DH Ratchet (Sym-DH). In this scenario, all Double Ratchet keys are leaked to the server. We assume a pre-shared secret double key dk that is not part of the leaked state. Our first construction uses this dk to define an anamorphic channel: Alice replaces some of her DH keys with fake keys that encode covert messages yet remain indistinguishable from honestly generated keys. Bob, who also knows dk, can recover the anamorphic messages from the received DH keys, while a server that only observes the full Double Ratchet state (but not dk) cannot detect or decode the covert channel.

Case 2: Asymmetric using DH ratchet (Asym-DH). The second scenario removes the need for a pre-shared symmetric double key. Instead, it relies on an asymmetric anamorphic key pair: Alice embeds covert messages into fake DH keys using Bob's double public key, and Bob recovers them using the corresponding double secret key. Intuitively, Asym-DH parallels Sym-DH, but instantiates the anamorphic channel with public-key settings, which is suitable when publishing a long-term public key is feasible, while keeping the corresponding secret key hidden from the server.

4.4. Optimization

As discussed before, since recovering anamorphic messages requires modular exponentiations, the anamorphic message space $\mathcal{M}$ must be small. We now present optimized variants of Algorithms 1 and 2 that use a precomputed lookup table, which is also used in our implementation. Intuitively, the optimized algorithms optimize the anamorphic decryption by replacing an exhaustive search with a single group operation and a table lookup. We successfully tested the feasibility of this optimisation for values up to 32 bits. However, it can be extended further, since the main computational effort lies in precomputing the lookup table only once. The same lookup-table technique extends to the construction with a stronger security guarantee in a similar way. The following is a concrete instantiation over Curve25519.

TABLE 2: Notation used in Algorithms 5 and 6

Symbol	Description
G_E	Base point of the Edwards curve E .
P_E and Q_E	Points on the Edwards curve E .
$f_{E \rightarrow M}$	Map from Edwards curve to Montgomery representation. Note that this function provides the extra bit of information sign needed for inversion, since the Montgomery model does not retain sign information.
$f_{M \rightarrow E}$	Map from Montgomery to Edwards representation, using the supplied choice of sign for the Edwards point.
$\text{lsb}(\text{pk}_i)$	Least significant bit of pk_i .
$\text{lookup}(Q_E)$	Lookup table mapping an Edwards curve point to a message.

In Algorithm 5, each fake secret key is chosen as a PRF-derived scalar (keyed by dk and the previous public key) plus the anamorphic message. The corresponding public key is then selected so that its least significant bit determines the correct sign to invert back to the unique Edwards point. This ensures that the public keys are distributed as in the honest scheme, while encoding a small message-dependent offset in a reversible way.

Algorithm 6 uses dk to recompute and subtract the PRF-derived scalar from the recovered Edwards point, yielding a point of the form $\text{amsg} \cdot G_E$. Since we precomputed a lookup table mapping all candidate points $\text{amsg} \cdot G_E$ to their corresponding messages, the decryption of each anamorphic block requires only one scalar multiplication and a table lookup, without iterating over all possible messages. This makes, for example, a 32-bit anamorphic message space efficiently practical in our implementation. The correctness and security guaranties of the original construction remain unchanged.

Algorithm 5 fakeKeygen(dk, $\overrightarrow{\text{amsg}}, \ell$)

```
1: (sk1, pk1) = Keygen( $\lambda$ )
2: for  $i \in [2, \ell + 1]$  do
3:   ctr = 0
4:   repeat
5:     ski = PRF(dk, pki-1, ctr) + amsgi-1 mod  $p$ 
6:     PE = ski · GE
7:     (pki, sign) = fE→M(PE)
8:     ctr = ctr + 1
9:   until lsb(pki) = sign
10: end for
11: return {(ski, pki)}i=1ℓ+1
```

Algorithm 6 aDec(dk, PK)

```
1: PK = (pki)i∈[1, ℓ+1]
2: for  $i \in [2, \ell + 1]$  do
3:   ctr = 0
4:   repeat
5:     x = PRF(dk, pki-1, ctr)
6:     PE = fM→E(pki, lsb(pki))
7:     QE = PE - x · GE
8:     amsgi-1 = lookup(QE)
9:     ctr = ctr + 1
10:  until amsgi-1 ≠ ⊥
11: end for
12: return {amsgi}i=1ℓ
```

5. Covert Channel in Symmetric Ratchet

To embed hidden messages into the symmetric ratchet, we create an anamorphic channel based on the MAC key that remains unknown to the server. Only the intended receiver, who holds this key, can recover the hidden messages as part of the integrity verification. This construction, namely *Sym-MAC*, targets scenarios where *either* the root key *or* the private DH ratchet key is leaked to the server, but not both, so that the resulting chain, message, and MAC keys are never fully exposed. From the adversary's view, the MACs carrying hidden messages are indistinguishable from ordinary MACs. No pre-shared key or other public double key is required, as the anamorphic messages are embedded using MAC keys derived within the Double Ratchet.

5.1. Base Construction

We first describe two algorithms that specify how to generate *fake* MACs and how to extract hidden messages from them.

Algorithm 7 embeds an anamorphic message amsg ∈ $\mathcal{M}$ into a MAC tag under key mac_{key} by XORing amsg with an honestly computed MAC. Algorithm 8 recovers the anamorphic message amsg ∈ $\mathcal{M}$ by recomputing the honest MAC tag under mac_{key} and extracting amsg via XOR.

In this construction, we eliminate the small message-space constraint required by the covert channel construction in DH keys. Here, the anamorphic message space is $\mathcal{M} = \{0, 1\}^\lambda$, where λ is the MAC length. In our implementation,

Algorithm 7 fakeMAC(mac_{key}, ct, ad, amsg)

```
1: t = MAC(mackey, ct, ad)
2: mac = t ⊕ amsg
3: return mac
```

Algorithm 8 aDec(mac_{key}, ct, ad, mac)

```
1: t = MAC(mackey, ct, ad)
2: if mac = t then
3:   skip ▷ Not anamorphic
4: else
5:   amsg = t ⊕ mac
6:   return amsg
7: end if
```

we embed 64-bit hidden messages, matching the MAC size specified in the Signal Double Ratchet documentation.

To integrate this into the full Double Ratchet protocol, Alice and Bob perform the following steps to transmit one anamorphic message amsg along with one normal Signal message m .

- 1) Alice encrypts a normal Signal message m into ct = AES_{CBC}(aes_{key}, IV, m) and computes a tag as described in Algorithm 7 instead of the original mac.
- 2) Bob detects and recovers the anamorphic message amsg by running aDec as shown in Algorithm 8.

We now argue for the correctness and security of the construction. Correctness requires that, given the MAC key mac_{key}, the hidden message can always be recovered; this follows directly from the invertibility of the XOR operation.

Anamorphic security requires that MACs carrying hidden messages be indistinguishable from honestly generated MACs. This holds as long as mac_{key} remains secret and the underlying MAC produces pseudorandom tags, since XOR with a fixed message preserves their indistinguishability from random. As illustrated in Figure 1, secrecy of mac_{key} implies secrecy of the corresponding message key and, in turn, of the chain key. Because deriving the chain key requires both the DH output and the root key, the construction remains secure even if either the private ratchet keys or the root key is compromised, as long as they are not both leaked simultaneously.

Anamorphic Security. The construction achieves anamorphic security if for every PPT adversary $\mathcal{D}$ it holds that

$$\left| \Pr[\text{GReal}_{\text{MAC}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GANam}_{\text{aMAC}}^{\mathcal{D}}(\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

where the security games are defined as follows.

In the security games, the distinguisher is given either rk or dh_{out} according to the security model. We explicitly show the intermediate steps to generate aes_{key}, mac_{key} and IV as they depend on the previous steps and ensure that in both games they are from the same distribution. We show that without the full knowledge of rk and dh_{out}, the keys generated along the KDF key are computationally indistinguishable from the random values sampled from the same distribution. The hybrid arguments then show that the anamorphic game is indistinguishable from one where

all keys are uniformly sampled and MACs are generated with random MAC keys, which is finally shown to be indistinguishable from the real game.

GReal_{MAC}^D(λ) :

- 1) $rk \leftarrow \{0, 1\}^\lambda$
- 2) $(sk_A, pk_A) \leftarrow \text{Keygen}(\lambda)$
- 3) $(sk_B, pk_B) \leftarrow \text{Keygen}(\lambda)$
- 4) $dh_{out} \leftarrow \text{DH}(sk_A, pk_B)$
- 5) **return** $\mathcal{D}^{\mathcal{O}_{MAC}(\cdot, \cdot, \cdot)}(\lambda, \text{info})$ where
 $\mathcal{O}_{MAC}(\text{amsg}, \text{ad}, m):$
 $(rk_{new}, ck) \leftarrow \text{KDF}_{rk}(rk, dh_{out})$
 $(ck_{new}, mk) \leftarrow \text{KDF}_{ck}(ck)$
 $(aes_{key}, mac_{key}, IV) \leftarrow \text{KDF}_{mk}(mk)$
 $ct \leftarrow \text{AES}_{CBC}(aes_{key}, IV, m)$
return $aes_{key}, IV,$
 $\text{MAC}(mac_{key}, ct, ad)$

GAnam_{aMAC}^D(λ) :

- 1) $rk \leftarrow \{0, 1\}^\lambda$
- 2) $(sk_A, pk_A) \leftarrow \text{Keygen}(\lambda)$
- 3) $(sk_B, pk_B) \leftarrow \text{Keygen}(\lambda)$
- 4) $dh_{out} \leftarrow \text{DH}(sk_A, pk_B)$
- 5) **return** $\mathcal{D}^{\mathcal{O}_{aMAC}(\cdot, \cdot, \cdot)}(\lambda, \text{info})$ where
 $\mathcal{O}_{aMAC}(\text{amsg}, \text{ad}, m):$
 $(rk_{new}, ck) \leftarrow \text{KDF}_{rk}(rk, dh_{out})$
 $(ck_{new}, mk) \leftarrow \text{KDF}_{ck}(ck)$
 $(aes_{key}, mac_{key}, IV) \leftarrow \text{KDF}_{mk}(mk)$
 $ct \leftarrow \text{AES}_{CBC}(aes_{key}, IV, m)$
return $aes_{key}, IV,$
 $\text{fakeMAC}(mac_{key}, ct, ad, \text{amsg})$

In both games, info = dh_{out} or rk

Theorem 2. *The construction Sym-MAC defined in Section 5.1 satisfies anamorphic security if the Decisional Diffie-Hellman problem is hard in $\mathbb{G}$, MAC used in Algorithm 7 is modelled as a secure PRF and all KDFs involved are secure KDFs, as introduced in Section 2.1.*

Proof. In the following sequence of hybrid games let $\mathcal{D}$ be a PPT adversary and Q its total number of oracle queries.

Game 0: This is the anamorphic game $\text{GAnam}_{aMAC}^D(\lambda)$ in the security definition, where the oracle generates anamorphic MACs as described in Algorithm 7, returning $t = \text{MAC}(mac_{key}, ct, ad) \oplus \text{amsg}$.

Game 1: In this game, we replace the use of KDF_{rk} , KDF_{ck} and KDF_{mk} to generate ck , mk , aes_{key} , mac_{key} and IV with elements chosen uniformly at random.

Game 2: In this game, we replace the application of MAC in Algorithm 7 by the application of a uniformly random function $\mathcal{F} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.

Game 3: In this game, we define $\mathcal{F}$ iteratively by lazy sampling: each time $\mathcal{F}$ should be applied, we check if we already established a mapping for the given input a , i.e., $a \mapsto z$ is contained in some set $\mathcal{M}_{\mathcal{F}}$, and if this is the case z is the output. Otherwise, some fresh $z \leftarrow \{0, 1\}^\lambda$ is sampled uniformly and the mapping $a \mapsto z$ is added to $\mathcal{M}_{\mathcal{F}}$.

Game (4, j) for $j \in [0, Q]$: In this series of games we slightly modify Game 3 until we end up with Game 4, by modifying the way each of the Q applications of $\mathcal{F}$ are computed. We define Game (4, 0) to be identical to Game 3. In Game (4, j), the challenger handles the first j applications of $\mathcal{F}$ as follows: Instead of checking if the a mapping for the corresponding input is already defined in $\mathcal{M}_{\mathcal{F}}$, it just samples a uniform output and adds the corresponding mapping to $\mathcal{M}_{\mathcal{F}}$. Note that in Game (4, Q) all applications of $\mathcal{F}$ are replaced by uniformly sampling elements from $\{0, 1\}^\lambda$.

Game 5: In this game, we replace the XOR computation with a uniformly random element independent of amsg .

Game 6: In this game, the oracle applies a freshly sampled MAC key to ct and ad .

Game 7: In this game, we replace the uniformly random chosen aes_{key} , mac_{key} and IV by generating them using the KDFs again. This reverses the changes made in Game 1.

Game 8: This is the real game $\text{GReal}_{MAC}^D(\lambda)$ in the security definition, where the relevant keys are generated by KDFs and the oracle applies a freshly sampled MAC key to ct and ad .

Game 0 and Game 1 are computationally indistinguishable, by Lemma 1. Game 1 and Game 2 are computationally indistinguishable, as a distinguisher for them could be turned in to one for pseudorandom property of PRF. The games Game 2 and Game 3 are perfectly indistinguishable as Game 3 just uses an equivalent way to define $\mathcal{F}$. Game (4, 0) coincides with Game 3 by definition. The difference between games Game (4, j - 1) and Game (4, j) consists in how the output of the j -th application of $\mathcal{F}$ is computed. In Game (4, j - 1) we check whether the input $mac_{key_{j-1}}$ occurred before in $\mathcal{M}_{\mathcal{F}}$, while we do not perform this check in Game (4, j). Observing that all mac_{key_i} for $i \leq j - 1$ have been chosen uniformly at random from $\mathcal{K}$, the probability that $mac_{key_i} \mapsto z$, for some z , occurs in $\mathcal{M}_{\mathcal{F}}$ is upper bounded by $\frac{j-1}{2^\lambda}$. Thus, in total the probability that games Game (4, 0) and Game (4, Q) differ is upper bounded by $\frac{Q^2}{2^\lambda}$, which is negligible. Since XOR with any value is a bijection on $\{0, 1\}^\lambda$, the joint distribution of all oracle outputs is identical in Game (4, Q) and Game 5, even if some inputs repeat, because lazy sampling preserves consistency. Thus Game 4, Q and Game 5 are perfectly indistinguishable. Finally, note that Game (4, Q) and Game 6 are identical. The computational indistinguishability from Game 6 to Game 7 is again from Lemma 1, and Game 7 and Game 8 are identical. $\square$

Lemma 1. Without the full knowledge of the root key and the DH output, a PPT adversary $\mathcal{D}$ could not distinguish whether aes_{key} , mac_{key} and IV are generated through a KDF chain or sampled uniformly at random.

We defer the proof for this lemma to the Appendix B.

5.2. Stronger Security Guarantee – IND-CCA

Similar to our previous construction, here, we also require that security holds even against an adversary that ob-

tains ciphertexts generated in anamorphic mode and actively interacts with the protocol.

Intuitively, Alice embeds ℓ anamorphic messages, each of length λ , within $\ell + 1$ normal Signal messages. The $\ell + 1$ MAC tags are generated in a way that the first ℓ tags used in the symmetric ratchet to carry the corresponding covert payloads, and the final tag authenticates the preceding transcript to provide an integrity check.

We update the fake MAC algorithm (Algorithm 7) and the anamorphic message decryption algorithm (Algorithm 8) to Algorithm 9 and Algorithm 10, respectively. We add an auxiliary PRF (same key space as MAC key) that maps the transcript to the tag.

Algorithm 9 $\text{fakeMAC}(\overrightarrow{\text{mac}}_{\text{key}}, \overrightarrow{\text{ct}}, \overrightarrow{\text{ad}}, \overrightarrow{\text{msg}})$

```

1: for  $i \in [1, \ell]$  do
2:    $\text{mac}_i = \text{MAC}(\text{mac}_{\text{key}_i}, \text{ct}_i, \text{ad}_i) \oplus \text{msg}_i$ 
3: end for
4:  $T = \text{mac}_1 \parallel \dots \parallel \text{mac}_\ell$ 
5:  $\text{mac}_{\ell+1} = \text{PRF}(\text{mac}_{\text{key}_{\ell+1}}, T)$ 
6: return  $\overrightarrow{\text{mac}}$ 

```

Algorithm 10 $\text{aDec}(\overrightarrow{\text{mac}}_{\text{key}}, \overrightarrow{\text{ct}}, \overrightarrow{\text{ad}}, \overrightarrow{\text{mac}})$

```

1:  $\overrightarrow{t} = \text{MAC}(\overrightarrow{\text{mac}}_{\text{key}}, \overrightarrow{\text{ct}}, \overrightarrow{\text{ad}})$ 
2: if  $\text{mac}_i = t_i$  for  $i \in [1, \ell + 1]$  then
3:   skip ▷ Not anamorphic
4: else
5:   for  $i \in [1, \ell]$  do
6:      $\text{msg}_i = \text{mac}_i \oplus t_i$ 
7:   end for
8:    $T = t_1 \parallel \dots \parallel t_\ell$ 
9:    $\sigma = \text{PRF}(\text{mac}_{\text{key}_{\ell+1}}, T)$ 
10:  if  $\sigma = \text{mac}_{\ell+1}$  then
11:    return  $\overrightarrow{\text{msg}}$ 
12:  else
13:    return  $\perp$ 
14:  end if
15: end if

```

Alice and Bob perform the following steps to transmit ℓ anamorphic messages $\overrightarrow{\text{msg}}$ within $\ell + 1$ normal Signal messages $\overrightarrow{m}$.

- 1) Alice encrypts $\ell + 1$ normal Signal messages $\overrightarrow{m}$ into $\overrightarrow{\text{ct}} = \text{AES}_{\text{CBC}}(\overrightarrow{\text{aes}}_{\text{key}}, \overrightarrow{\text{IV}}, \overrightarrow{m})$ where $\overrightarrow{\text{aes}}_{\text{key}}$ and $\overrightarrow{\text{IV}}$ are generated honestly along the KDF chain.
- 2) Alice then computes the tags as described in Algorithm 9 and uses them as MACs corresponding to the messages instead of the original $\overrightarrow{\text{mac}}$.
- 3) Bob detects and recovers the anamorphic message msg by running aDec as shown in Algorithm 10.

We defer the security proof to Appendix C.

6. Performance

We implement prototypes of anamorphic messaging constructions [Ano25] in the official Signal application [Sig25c] using Rust, Java, and Kotlin. We empirically evaluate the impact of these implementations across different covert bit-rate settings on the overall performance of the Signal application, both in the official Signal-Android client [Sig25b] and libSignal [Sig25a], the platform-agnostic library used by the official Signal clients which is mainly written in Rust.

We run our experiments on an Android device running Android 14 with a Qualcomm Snapdragon 888 processor and 8 GB of RAM, and on Android Studio emulators hosted in a 2023 MacBook Pro laptop on an M2 Pro processor with 16GB of RAM.

Tables 3 presents performance results for experiments conducted on a real Android device and an Android emulator. We report the average encryption and decryption times for random messages using each anamorphic messaging construction, alongside the original Signal protocol.

We also evaluate the average worst-case decryption time, which occurs when a message does *not* contain an anamorphic payload. In addition, we measure the round-trip time on the recipient side, defined as the duration between receiving the message from the server and sending the corresponding delivery receipt back. This metric is important for assessing potential side-channel attack vectors, as the time taken to generate and send the receipt could reveal to the server whether the message was anamorphic.³

As decryption is the most computationally intensive part of our constructions and a key performance indicator, we evaluate the decryption time in both Signal-Android and libsignal to gain a comprehensive view of performance. To compute the function aDec in our constructions (Sym-DH, Asym-DH, and Sym-MAC), we implemented the base variants. Moreover, we implemented the optimized variants of the construction (A)Sym-DH on the Android emulator to show their huge performance advantage. Note that the PRFs in Sym-DH and Asym-DH are instantiated using HMAC-SHA256.

In general, message encryption times are not affected by the generation of anamorphic messages. Looking at libsignal on the real device, the original Signal protocol takes approximately 1.53 ms to encrypt a message, while all anamorphic constructions require nearly the same time to encrypt an anamorphic message, regardless of its size. Similar results can be observed on the Android emulator, as shown in Table 3.

The decryption time is affected by the type of construction. This is expected, as the decryption process might involve iterative operations, particularly the discrete logarithm

3. One could avoid sending the "delivery" message at all. In such a case, the DH ratchet would not be updated (since the delivery message carries a fresh public key) and the sender would need to wait for a message reply from the receiver before sending a new anamorphic message in the Sym-DH and Asym-DH constructions. In addition, not sending a delivery message might appear suspicious to the server, as sending a delivery message is standard in Signal.

TABLE 3: Performance (in milliseconds) on Android real device (top) and Android emulator (bottom).

Construction	Anamorphic length	Signal-Android				libSignal		
		Decryption		Round-trip		Encryption	Decryption	
		Avg	Worst	Avg	Worst	Avg	Avg	Worst
Android real device								
Original	None	1.56	—	20.5	—	1.53	1.42	—
Base (A)Sym-DH	16-bit	822.8	1622.2	841.48	1673.28	1.71	811.34	1617.41
Sym-MAC	64-bit	1.57	—	20.6	—	1.53	1.40	—
Android emulator								
Original	None	1.50	—	120.10	—	1.11	1.11	—
Base (A)Sym-DH	16-bit	438.2	888.2	567.3	1010.2	1.81	440.3	883.2
Optimized	16-bit	7.01	14.12	149.96	150.14	1.13	5.77	13.20
(A)Sym-DH	32-bit	7.22	65.10	150.32	200.18	1.32	5.43	65.88
Sym-MAC	64-bit	1.51	—	120.15	—	1.11	1.12	—

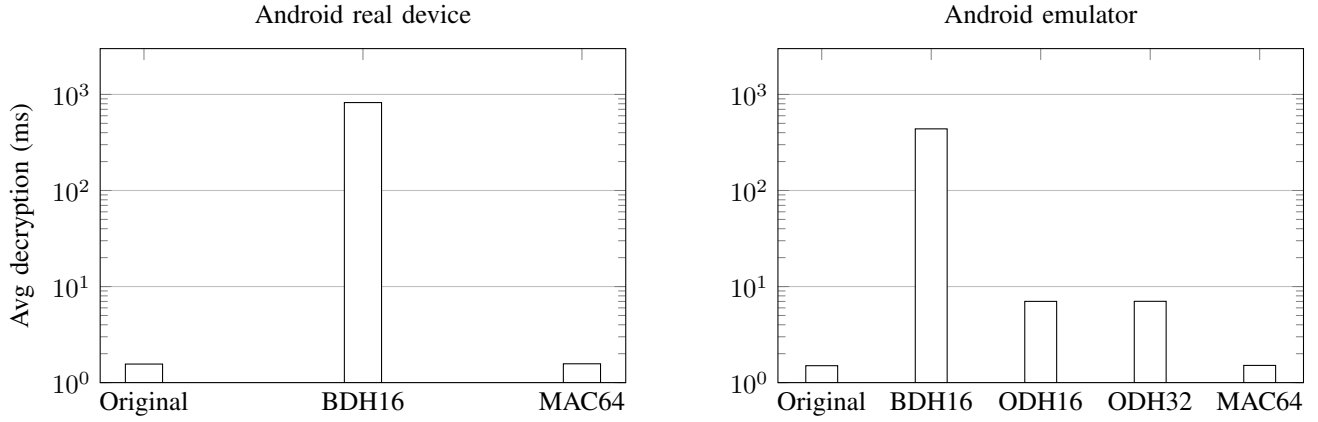


Figure 4: Average decryption latency in the Android emulator and Android real device measured in Signal Android library, grouped by construction and anamorphic bit-length (log scale). BDH16, MAC64, ODH16, and ODH32 stand for Base (A)Sym-DH 16-bit, Sym-MAC 64-bit, Optimized (A)Sym-DH 16-bit, and Optimized (A)Sym-DH 32-bit, respectively.

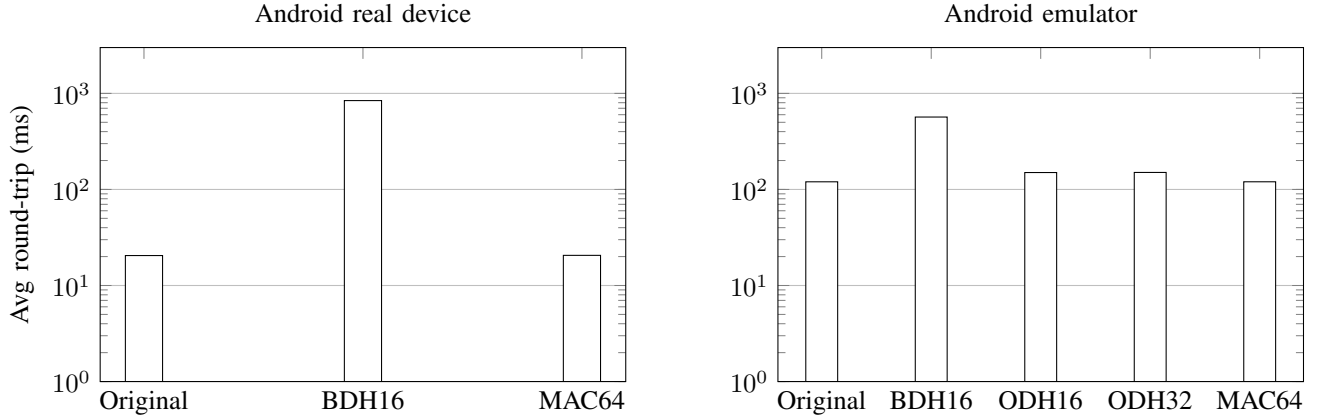


Figure 5: Average round-trip latency in the Android emulator and Android real device measured in Signal Android library, grouped by construction and anamorphic bit-length (log scale).

computation required to recover the anamorphic message in Sym-DH and Asym-DH constructions. However, there is no significant difference, on average, within the optimized (A)Sym-DH construction, nor between the decryption in the

core libSignal library and the Signal-Android client.

The round-trip time, as measured in the Signal-Android code, varies considerably only in the base (A)Sym-DH construction, while it does not deviate from the original

Signal implementation in either the Sym-MAC or the optimized (A)Sym-DH constructions. Therefore, the Sym-MAC construction is the most efficient according to all indicators. However, it supports the weakest threat model. The optimized (A)Sym-DH construction supports the strongest threat model and performs within the same order of magnitude as the original Signal construction. The base (A)Sym-DH construction takes significantly longer than the original Signal protocol, as the receiver must decrypt up to 16 bits of anamorphic information, involving multiple scalar multiplications, as shown in Figures 4 and 5. This construction is therefore suitable only when a lookup table cannot be maintained. In this case, to mitigate side-channel risks, the receiver can send the delivery receipt after a fixed delay matching the average round-trip time in the original implementation, rather than waiting for successful decryption. However, the sender should transmit each message using an interval longer than the worst-case round-trip time—approximately 1600 ms on the real device and 1000 ms on the emulator.

7. Related Work

The foundations of modern secure messaging go back to Off-the-Record Messaging (OTR) [BGB04], which introduced forward secrecy and deniability. Later, TextSecure [Mar13] extended these ideas with a structured cryptographic framework that evolved into the Signal protocol [PM16a], now considered the standard for private messaging. At the heart of Signal is the Double Ratchet algorithm, which has been formally studied in multiple works. Alwen et al. [ACD19] presented a modular analysis of its components; Poettering and Rösler [PR18] investigated its compromise-resilience; and Cohn-Gordon et al. [CGCD⁺20] provided symbolic and computational proofs of its E2EE guarantees. Bienstock et al. [BFG⁺22] proposed a more complete Universally Composable framework, introducing a minimal extension called the Triple Ratchet, which captures stronger adversarial models with no added cost.

More recently, Teng and Rasmussen [TR24] showed that key verification in Signal can be automated within a modified protocol, removing the need for out-of-band user checks such as safety-number comparison or QR scanning and strengthening resistance to man-in-the-middle attacks. Martiny et al. [MKA⁺21] analyzed the Sealed Sender mechanism in Signal and demonstrated how statistical disclosure attacks could compromise its anonymity guarantees.

In parallel, the Signal protocol has been extended to resist emerging threats such as quantum adversaries. PQXDH [KS23] replaces the X3DH handshake with a post-quantum alternative, while Dodis et al. [DJK⁺25] introduced a Triple Ratchet protocol that incorporates a more efficient PQ-ratchet and erasure coding, addressing bandwidth limitations in hybrid-secure settings. In a related development within the quantum-secure Signal, the ML-KEM Braid protocol [Sch25] was proposed, based on the ideas of [ADJ⁺25]. It is a Sparse Continuous Key Agreement (SCKA) built on ML-KEM that lets two parties derive a sequence of post-quantum

shared secrets with forward secrecy and post-compromise security. The Classical Continuous Key Agreement in the Double Ratchet emits a fresh secret each round trip, whereas SCKA handles bandwidth constraints by sending large KEM data in pieces so new secrets may arrive over multiple round trips, and the resulting keys can feed a Double Ratchet for secure messaging. Additionally, efforts have been made to enhance secure messaging protocols like Signal’s X3DH and Double Ratchet with subversion resilience, employing techniques such as *Reverse Firewalls*, which protect against attacks that modify cryptographic algorithms, ensuring that users can detect tampering even with compromised implementations. [DMSDT25].

The robustness of secure messaging systems, particularly Signal, lies in its ability to guarantee security and privacy even in adversarial environments. However, as the landscape of cryptographic security evolves, new threats such as deployment-level compromises emerge, challenging the reliability of E2EE. In light of these challenges, anamorphic encryption offers a promising approach to preserve security in the presence of surveillance that compromises E2EE.

Anamorphic encryption was first introduced by Persiano et al. [PPY22] as a method for enabling covert communication when a surveillance authority can decrypt messages, but a cooperating sender and receiver can still exchange hidden content embedded inside normal-looking ciphertexts. Since its introduction, several works have proposed stronger and more robust definitions of anamorphic encryption, adapting it to current cryptographic primitives [CGM24], [CGM25], [KPP⁺23], [BGH⁺24], [WCHY23].

One of the early developments was by Wang et al. [WCHY23], who introduced the notion of ℓ -sender-anamorphic encryption, formalizing secure constructions for pseudo-random public key encryption (PKE) schemes. Following this work, Banerjee et al. [BPRS25] introduced a framework for public-key anamorphic μ -message extensions, embedding covert messages within multiple ciphertexts of CPA-secure PKE schemes, without relying on randomness recovery.

Another research domain that has a goal on the opposite side of anamorphic encryption is Anamorphic-Resistant Encryption (ARE). Dodis and Goldin [DG25] and Carnemolla et al. [CCGM25] propose encryption schemes that specifically block covert communication, opposing the core goal of anamorphic encryption. ARE schemes aim to make covert channels either slow or entirely impractical, thereby undermining anamorphic encryption’s ability to enable hidden communication in the presence of surveillance that breaks E2EE. Avitabile et al. [ABG⁺25] present a practical Anamorphic-Resistant Encryption (ARE) scheme that addresses key limitations of previous constructions [DG25], [CCGM25], notably by eliminating the reliance on random oracles and indistinguishability obfuscation. Their scheme, based on Extremely Lossy Functions (ELFs) and Unique Non-Interactive Zero-Knowledge proofs (UNIZKs), ensures that covert communication is limited to a logarithmic number of bits, effectively hindering the viability of anamorphic encryption for transmitting long anamorphic messages.

In recent years, research on anamorphism in application-level cryptographic protocols has gained considerable momentum. Giustolisi et al. [GRL25] adapted the ElGamal-based construction from Wang et al. [WCHY23] to electronic voting systems, enabling voters to embed hidden preferences within ballots that are indistinguishable from legitimate ones, even against malicious authorities.

More recently, Gligoroski et al. [GRJ25] explored the application of anamorphic encryption in conversational settings, embedding covert messages within text exchanged over public chat channels. Their framework introduced an LLM-based embedding function, enabling both symmetric and public key encryption schemes (e.g., password-authenticated encryption and elliptic curve DH key exchange with ephemeral keys) to operate over natural language outputs. This work opened new directions for covert communication through LLM-generated text.

Despite its theoretical foundations, anamorphic encryption has seen limited exploration in practical applications. Secure messaging systems like Signal are often targeted by adversaries or regulatory efforts seeking to undermine E2EE. Legislative proposals for lawful access to encrypted data [Eur25] highlight the need for secure messaging protocols that remain functional under E2EE compromise. Our work builds on the principles of anamorphic encryption, applying it to the Signal Double Ratchet protocol to enable covert communication even if the confidentiality of E2EE is intentionally broken.

8. Conclusions

In this work, we presented two constructions on anamorphic channels for the Signal Double Ratchet protocol: a DH-based channel that embeds payloads in DH key pairs and a MAC-based channel that embeds payloads in authentication tags of the symmetric ratchet. Our designs remain compatible with existing Signal-style deployments, require no server changes, and operate even when one or more Double Ratchet keys are disclosed (for example as a result of a surveillance mechanism). Our analysis and implementations show that the DH-based and MAC-based channels can efficiently carry at least 32 and 64 hidden bits per Signal message, respectively. To the best of our knowledge, this is the first practical instantiation of anamorphic encryption on a deployed system, the Signal Double Ratchet, that has impact on billions of people.

Several policymakers and governments have pushed Signal and other E2EE messaging platforms to enable surveillance by breaking E2EE confidentiality to snoop on messages. Our results indicate that even if decryption keys are exposed, covert channels that preserve normal protocol behavior can still exist.

A key direction for future work concerns the potential deployment of Anamorphic-Resistant Encryption schemes. If ARE systems become efficient and widely used, it will be important to understand whether some form of anamorphism can still be maintained, and under what leakage or functionality constraints. In particular, studying ARE-style

defenses in the context of the Signal protocol would clarify the minimal conditions under which covert channels remain possible.

Finally, our approach is worth investigating in protocols that reuse Double Ratchet-style building blocks, including recent post-quantum proposals such as Triple Ratchet [DJK⁺25] and Apple’s PQ3 [App24]. Beyond messaging, similar embeddings may apply to other E2EE services, including file sharing and real-time audio and video, offering a clearer path to supporting E2EE even under key-disclosure pressure.

References

- [ABG⁺25] Gennaro Avitabile, Vincenzo Botta, Emanuele Giunta, Marcin Mielniczuk, and Francesco Migliaro. The malice of elfs: Practical anamorphic-resistant encryption without random oracles. *Cryptology ePrint Archive*, 2025.
- [ACD19] Joël Alwen, Sandro Coretti, and Yevgeniy Dodis. The double ratchet: security notions, proofs, and modularization for the signal protocol. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 129–158. Springer, 2019.
- [ADJ⁺25] Benedikt Auerbach, Yevgeniy Dodis, Daniel Jost, Shuichi Katsumata, and Rolfe Schmidt. How to compare bandwidth constrained two-party secure messaging protocols: A quest for a more efficient and secure post-quantum protocol. In *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [Ano25] Anonymised. AnamorphicSignal implementations. <https://anonymous.4open.science/r/AnamorphicSignal-MAC64>, 2025.
- [App24] Apple Security Engineering and Architecture (SEAR). iMessage with PQ3: The new state of the art in quantum-secure messaging at scale. <https://security.apple.com/blog/imessage-pq3/>, 2024. Accessed: 07/2025.
- [BBC23] BBC news. Signal would walk from UK if Online Safety Bill undermined encryption, 2023. Accessed: 08/2025.
- [BFG⁺22] Alexander Bienstock, Jaiden Fairoze, Sanjam Garg, Pratyay Mukherjee, and Srinivasan Raghuraman. A more complete analysis of the signal double ratchet algorithm. In *Annual International Cryptology Conference*, pages 784–813. Springer, 2022.
- [BGB04] Nikita Borisov, Ian Goldberg, and Eric Brewer. Off-the-record communication, or, why not to use pgp. In *Proceedings of the 2004 ACM Workshop on Privacy in the Electronic Society*, pages 77–84, 2004.
- [BGH⁺24] Fabio Banfi, Konstantin Gegier, Martin Hirt, Ueli Maurer, and Guilherme Rito. Anamorphic encryption, revisited. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–32. Springer, 2024.
- [BPRS25] Shalini Banerjee, Tapas Pal, Andy Rupp, and Daniel Slamanig. Simple public key anamorphic encryption and signature using multi-message extensions. *Cryptology ePrint Archive*, 2025.
- [CCGM25] Davide Carnemolla, Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic resistant encryption: the good, the bad and the ugly. *Cryptology ePrint Archive*, 2025.
- [CGCD⁺20] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the signal messaging protocol. *Journal of Cryptology*, 33(4):1914–1983, 2020.

- [CGM24] Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic encryption: New constructions and homomorphic realizations. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 33–62. Springer, 2024.
- [CGM25] Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Generic anamorphic encryption, revisited: New limitations and constructions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 275–303. Springer, 2025.
- [CJSV22] Ran Canetti, Palak Jain, Marika Swanberg, and Mayank Varia. Universally composable end-to-end secure messaging. In *Annual International Cryptology Conference*, pages 3–33. Springer, 2022.
- [CRT24] Daniel Collins, Doreen Riepel, and Si An Oliver Tran. On the tight security of the double ratchet. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4747–4761, 2024.
- [cyb25] cyberdaily.au. Signal threatens to withdraw from Australia as government pushes for backdoor data access, 2025. Accessed: 08/2025.
- [DG25] Yevgeniy Dodis and Eli Goldin. Anamorphic-resistant encryption; or why the encryption debate is still alive. *Cryptology ePrint Archive*, 2025.
- [dig25] digwatch. Sweden considers law requiring encrypted messaging backdoors, Signal threatens to exit, 2025. Accessed: 08/2025.
- [DJK⁺25] Yevgeniy Dodis, Daniel Jost, Shuichi Katsumata, Thomas Prest, and Rolfe Schmidt. Triple ratchet: A bandwidth efficient hybrid-secure signal protocol. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 302–331. Springer, 2025.
- [DMSDT25] Yevgeniy Dodis, Bernardo Magri, Noah Stephens-Davidowitz, and Yiannis Tselekounis. Guarding the signal: Secure messaging with reverse firewalls. *Cryptology ePrint Archive*, 2025.
- [EU 25] EU Presidency. Proposal for a Regulation of the European Parliament and of the Council laying down rules to prevent and combat child sexual abuse, 2025. Accessed: 10/2025.
- [Eur25] European Commission. On ProtectEU: A European Internal Security Strategy, 2025. Accessed: 07/2025.
- [GRJ25] Danilo Gligoroski, Mayank Raikwar, and Sonu Kumar Jha. An llm framework for cryptography over chat channels. *arXiv preprint arXiv:2504.08871*, 2025.
- [GRL25] Rosario Giustolisi, Mohammadamin Rakeei, and Gabriele Lenzini. Anamorphic voting: Ballot freedom against dishonest authorities. *Cryptology ePrint Archive*, 2025.
- [KBB17] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In *2017 IEEE European symposium on security and privacy (EuroS&P)*, pages 435–450. IEEE, 2017.
- [KCKG24] Mallory Knodel, Sofia Celi, Olaf Kolkman, and Gurshabad Grover. Definition of end-to-end encryption. *Cryptology ePrint Archive*, Paper 2024/2085, 2024.
- [KPP⁺23] Mirosław Kutylowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. The self-anti-censorship nature of encryption: On the prevalence of anamorphic cryptography. *Proceedings on Privacy Enhancing Technologies*, 2023(4):170–183, 2023.
- [Kra10] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. *IACR Cryptol. ePrint Arch.*, page 264, 2010.
- [KS23] Ehren Kret and Rolfe Schmidt. The pqxdh key agreement protocol. <https://signal.org/docs/specifications/pqxdh/pqxdh.pdf>, 2023.
- [Mar13] Moxie Marlinspike. Advanced cryptographic ratcheting. <https://signal.org/blog/advanced-ratcheting/>, 2013.
- [MKA⁺21] Ian Martiny, Gabriel Kaptchuk, Adam J Aviv, Daniel S Roche, and Eric Wustrow. Improving signal’s sealed sender. In *NDSS*, 2021.
- [PM16a] Trevor Perrin and Moxie Marlinspike. The double ratchet algorithm. <https://signal.org/docs/specifications/doubleratchet/>, 112(4), 2016.
- [PM16b] Trevor Perrin and Moxie Marlinspike. The x3dh key agreement protocol. <https://signal.org/docs/specifications/x3dh/>, 2016.
- [PPY22] Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Anamorphic encryption: private communication against a dictator. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 34–63. Springer, 2022.
- [PR18] Bertram Poettering and Paul Rösler. Towards bidirectional ratcheted key exchange. In *Annual International Cryptology Conference*, pages 3–32. Springer, 2018.
- [Sch25] Rolfe Schmidt. The ml-kem braid protocol. <https://signal.org/docs/specifications/mlkembraid/>, 2025.
- [Sig25a] Signal. libsignal. <https://github.com/signalapp/libsignal>, 2025. Github repository, Accessed: 05/2025.
- [Sig25b] Signal. Signal-android. <https://github.com/signalapp/Signal-Android>, 2025. Github repository, Accessed: 05/2025.
- [Sig25c] Signal. signalapp. <https://github.com/signalapp>, 2025. Github repository, Accessed: 05/2025.
- [TR24] Wil Liam Teng and Kasper Rasmussen. Defending against attack on the cloned: In-band active man-in-the-middle detection for the signal protocol. *arXiv preprint arXiv:2410.16098*, 2024.
- [VGIK20] Nihal Vatandas, Rosario Gennaro, Bertrand Ithurburn, and Hugo Krawczyk. On the cryptographic deniability of the signal protocol. In *International Conference on Applied Cryptography and Network Security*, pages 188–209. Springer, 2020.
- [WCHY23] Yi Wang, Rongmao Chen, Xinyi Huang, and Moti Yung. Sender-anamorphic encryption reformulated: Achieving robust and generic constructions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 135–167. Springer, 2023.

Appendix

1. IND-CCA Security for Covert Messaging in DH Ratchet

Definition 1 (IND-CCA for Covert Channel in DH Ratchet). Let $\mathcal{A}$ be a PPT algorithm. The security experiment for indistinguishability under adaptive chosen ciphertext attack is $\text{Exp}_{\text{DH}}^{\text{ind-cca}}$ shown in Figure 6. The advantage of $\mathcal{A}$ in the experiment is

$$\text{Adv}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr[\text{Exp}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A}) \rightarrow 1] - \frac{1}{2} \right|$$

$\text{Exp}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A})$
1 : $\text{dk} \leftarrow \mathbb{S}\{0, 1\}^\lambda$
2 : $S = \emptyset$
3 : $b \leftarrow \mathbb{S}\{0, 1\}$
4 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{fKGen}_b}(\cdot, \cdot), \mathcal{O}_{\text{aDec-DH}}(\cdot)}(1^\lambda, 1^\ell)$
5 : if $b' = b$ then
6 : return 1
7 : else
8 : return 0
$\mathcal{O}_{\text{fKGen}_b}(\overrightarrow{\text{ams}}_0, \overrightarrow{\text{ams}}_1)$
1 : $C \leftarrow \text{fakeKeygen}(\text{dk}, \overrightarrow{\text{ams}}_b)$
2 : $S = S \cup \{C\}$
3 : return C
$\mathcal{O}_{\text{aDec-DH}}(C^*)$
1 : if $C^* \in S$
2 : return $\perp$
3 : else
4 : parse $C = \{(\text{pk}_i, \text{sk}_i)\}_{i=1}^{\ell+2}$
5 : return $\text{aDec}(\text{dk}, \{\text{pk}_i\}_{i=1}^{\ell+2})$

Figure 6: Security experiment for IND-CCA for Covert Channel in DH Ratchet

Theorem 3. If PRFs used in Appendix A are secure PRFs, for every PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

Proof. Let $\Pr[\text{Game } i]$ be the adversary's success probability in game Game i , Q be the total number of oracle queries. Game 0: This game is exactly the IND-CCA experiment and oracle. Hence,

$$\text{Adv}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr[\text{Game } 0] - \frac{1}{2} \right|$$

Game 1: In this game, we replace PRF^* in the decryption algorithm aDec with a random function $\mathcal{F} : \{0, 1\}^* \mapsto \mathbb{Z}_p$. Let $\mathcal{B}_1$ be a PPT adversary against PRF security. By the security of PRF, we have

$$|\Pr[\text{Game } 0] - \Pr[\text{Game } 1]| \leq \text{Adv}^{\text{prf}}(\mathcal{B}_1)$$

Game 2: In this game, we define $\mathcal{F}$ iteratively by lazy sampling: we maintain a table for $\mathcal{F}$, and on input T : if T is already in the table, return the stored $\text{pk}_{\ell+2}$, otherwise sample $\text{pk}_{\ell+2} \leftarrow \mathbb{Z}_p$ uniformly, store and return it. Game 1 and Game 2 use equivalent methods to define $\mathcal{F}$, and thus

$$\Pr[\text{Game } 1] = \Pr[\text{Game } 2]$$

Game 3: We first define a Forge event: the aDec algorithm accepts a *fresh* ciphertext. In Game 2, for any fresh T^* , $\text{pk}_{\ell+2} = \text{PRF}^*(\text{dk}, T^*)$ is uniform and hidden. Acceptance requires guessing $\text{pk}_{\ell+2} = \text{sk}_{\ell+2} \cdot G$. In a group of prime order p , this succeeds with probability $\frac{1}{p}$ per query, and thus

$$\Pr[\text{Forge}] \leq \frac{Q}{p}.$$

Now in this game, it is identical to Game 2 except that on the first accepting fresh decryption query, the decryption oracle directly aborts. Then,

$$|\Pr[\text{Game } 2] - \Pr[\text{Game } 3]| \leq \Pr[\text{Forge}]$$

Game 4: In this game, we replace $\{(\text{pk}_i)\}_{i=1}^{\ell+1}$ with fresh DH pairs.

Let $\mathcal{B}_2$ be a PPT adversary against anamorphic security we defined before, then we have

$$|\Pr[\text{Game } 3] - \Pr[\text{Game } 4]| \leq \text{Adv}_{\text{DH}}^{\text{ana}}(\mathcal{B}_2).$$

Game 5: In the final game, all the $\{(\text{pk}_i)\}_{i=1}^{\ell+2}$ has been replaced by random values, and the decryption oracle gives no information. This is identical to the previous Game 4 and the adversary's entire view does not depend on the hidden bit b . Therefore, we have

$$\Pr[\text{Game } 5] = \Pr[\text{Game } 4] = \frac{1}{2}.$$

Combining the hops, we derive

$$\text{Adv}_{\text{DH}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{Adv}^{\text{prf}}(\mathcal{B}_1) + \text{Adv}_{\text{DH}}^{\text{ana}}(\mathcal{B}_2) + \frac{Q}{p}$$

which is negligible. $\square$

2. Proof for Lemma 1

Proof. To prove this, we define the following two games and show that for every PPT adversary $\mathcal{D}$ it holds that

$$|\Pr[\text{GReal}_{\text{key}}^{\mathcal{D}}(\lambda) = 1] - \Pr[\text{GRand}_{\text{rkey}}^{\mathcal{D}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

For simplicity, in the experiments, we show the KDF chain of only one KDF_{rk} block, one KDF_{ck} block and one KDF_{mk} block. When the KDF_{rk} block is fixed, the longer KDF chain leads to the more iterations of the games involving KDF_{ck} and KDF_{mk} where uniformly random ck and mk are used as inputs respectively.

$\text{GReal}_{\text{key}}^{\mathcal{D}}(\lambda) :$

- 1) $\text{rk} \leftarrow \{0, 1\}^\lambda$
- 2) $(\text{sk}_A, \text{pk}_A) \leftarrow \text{Keygen}(\lambda)$
- 3) $(\text{sk}_B, \text{pk}_B) \leftarrow \text{Keygen}(\lambda)$
- 4) $\text{dh}_{\text{out}} \leftarrow \text{DH}(\text{sk}_A, \text{pk}_B)$
- 5) **return** $\mathcal{D}^{\mathcal{O}_{\text{key}}(\cdot)}(\lambda, \text{info})$ where
 $\mathcal{O}_{\text{key}}(\text{info}) :$
 - $\text{prf}_{\text{key}} \leftarrow \text{HMAC}(\text{info}, \hat{\text{info}})$
 - $(\text{rk}_{\text{new}}, \text{ck}) \leftarrow \text{HKDF}_{\text{expand}}(\text{prf}_{\text{key}})$
 - $(\text{ck}_{\text{new}}, \text{mk}) \leftarrow \text{KDF}_{\text{ck}}(\text{ck})$
 - return** $\text{KDF}_{\text{mk}}(\text{mk})$

$\text{GRand}_{\text{rkey}}^{\mathcal{D}}(\lambda) :$

- 1) $\text{rk} \leftarrow \{0, 1\}^\lambda$
- 2) $(\text{sk}_A, \text{pk}_A) \leftarrow \text{Keygen}(\lambda)$
- 3) $(\text{sk}_B, \text{pk}_B) \leftarrow \text{Keygen}(\lambda)$
- 4) $\text{dh}_{\text{out}} \leftarrow \text{DH}(\text{sk}_A, \text{pk}_B)$
- 5) **return** $\mathcal{D}^{\mathcal{O}_{\text{key}}(\cdot)}(\lambda, \text{info})$ where
 $\mathcal{O}_{\text{rkey}}(\text{info}) = (\text{aes}_{\text{key}}, \text{mac}_{\text{key}}, \text{IV}) \leftarrow \{0, 1\}^{3\lambda}$

In the games, info is either rk or dh_{out} and $\hat{\text{info}}$ is the other. We explicitly detail the extract-then-expand technique used by the HKDF KDF_{rk} in the games, which consists of two steps:

- 1) The extract function takes as input the salt rk and the input key material dh_{out} , producing a PRF key. This extract function is instantiated using HMAC with SHA-256.
- 2) The expand function takes the PRF key, auxiliary information, and the desired output length as input, and outputs the new root key and chain key. We omit the auxiliary information and length here, which will be instantiated by PRF.

In the following sequence of hybrid games let $\mathcal{D}$ be a PPT adversary and Q its maximum number of queries to each oracle.

Game 0: This is the real game $\text{GReal}_{\text{key}}^{\mathcal{D}}(\lambda)$ in the definition, where the oracle generates all the keys as in the symmetric ratchet.

Game 1 **if** $\text{info} = \text{dh}_{\text{out}}$: In this game, we replace $\hat{\text{info}}$ with a uniformly random element u from $\mathbb{G}$.

Game 2: In this game, we replace HMAC by the application of a uniformly random function $\mathcal{F}_0 : \mathbb{G} \mapsto \{0, 1\}^\lambda$ if $\text{info} := \text{dh}_{\text{out}}$, otherwise $\mathcal{F}_0 : \{0, 1\}^\lambda \mapsto \{0, 1\}^\lambda$.

Game 3: In this game, we define $\mathcal{F}_0$ by lazy sampling: each time $\mathcal{F}_0$ should be applied, we check if we already established a mapping for the given input a , i.e., $a \mapsto z$ is contained in some set $\mathcal{M}_{\mathcal{F}_0}$, and if this is the case z is the output. Otherwise, some fresh $z \leftarrow \{0, 1\}^\lambda$ is sampled uniformly and the mapping $a \mapsto z$ is added to $\mathcal{M}_{\mathcal{F}_0}$.

Game (4, j) **for** $j \in [0, Q]$: In this series of games we slightly modify Game 3 until we end up with Game 5, by modifying the way each of the Q applications of $\mathcal{F}_0$ are computed. We define Game (4, 0) to be identical to Game 3. In Game (4, j), the challenger handles the first j applications of $\mathcal{F}$ as follows: Instead of checking if the

mapping for the corresponding input is already defined in $\mathcal{M}_{\mathcal{F}}$, it just samples a uniform output and adds the corresponding mapping to $\mathcal{M}_{\mathcal{F}_0}$. Note that in Game (4, Q) all applications of $\mathcal{F}_0$ are replaced by uniformly sampling elements from $\{0, 1\}^\lambda$.

Game 5: In this game, prf_{key} is sampled uniformly at random.

Game 6: In this game, we replace the expand algorithm by the application of a uniformly random function $\mathcal{F}_1 : \{0, 1\}^\lambda \mapsto \{0, 1\}^{2\lambda}$.

Game 7: In this game, we define $\mathcal{F}_1$ by lazy sampling, in the same manner as Game 3.

Game (8, j) **for** $j \in [0, Q]$: In this series of games we slightly modify Game 6 until we end up with Game , by modifying the way each of the Q applications of $\mathcal{F}_0$ is computed. These steps are in the same manner as in Game 8 and in Game (8, Q), all applications of $\mathcal{F}_1$ are replaced by uniformly random elements from $\{0, 1\}^{2\lambda}$.

Game 9: In this game, $(\text{rk}_{\text{new}}, \text{ck})$ is sampled uniformly at random.

Game 10: In this game, we replace KDF_{ck} by the application of a uniformly random function $\mathcal{F}_2 : \{0, 1\}^\lambda \mapsto \{0, 1\}^{2\lambda}$.

Game 11: In this game, we define $\mathcal{F}_2$ by lazy sampling, in the same manner as Game 3.

Game (12, j) **for** $j \in [0, Q]$: In this series of games we slightly modify Game 11 until we end up with Game , by modifying the way each of the Q applications of $\mathcal{F}_2$ is computed. These steps are in the same manner as in Game 4 and in Game (12, Q), all applications of $\mathcal{F}_2$ are replaced by uniformly random elements from $\{0, 1\}^{2\lambda}$.

Game 13: In this game, $(\text{ck}_{\text{new}}, \text{mk})$ is sampled uniformly at random.

Game 14: In this game, we replace KDF_{mk} by the application of a uniformly random function $\mathcal{F}_3 : \{0, 1\}^\lambda \mapsto \{0, 1\}^{3\lambda}$.

Game 15: In this game, we define $\mathcal{F}_3$ by lazy sampling, in the same manner as Game 3.

Game (16, j) **for** $j \in [0, Q]$: In this series of games we slightly modify Game 15 until we end up with Game , by modifying the way each of the Q applications of $\mathcal{F}_3$ is computed. These steps are in the same manner as in Game 4 and in Game (16, Q), all applications of $\mathcal{F}_3$ are replaced by uniformly random elements from $\{0, 1\}^{3\lambda}$.

Game 17: In this game, $(\text{aes}_{\text{key}}, \text{mac}_{\text{key}}, \text{IV})$ is sampled uniformly at random, which is the ideal game $\text{GRand}_{\text{rkey}}^{\mathcal{D}}$.

Game 0 and Game 1 (if there is) are computationally indistinguishable as a distinguisher for them could be turned into one for solving DDH problem in $\mathbb{G}$. Game 0 and Game 2 are computationally indistinguishable, as a distinguisher for them could be turned into one for the PRF security of HMAC. Game 2 and Game 3 are perfectly indistinguishable as Game 3 just uses an equivalent way to define $\mathcal{F}_0$. Game (4, 0) coincides with Game 3 by definition. The difference between games Game (4, $j - 1$) and Game (4, j) consists in how the output of the j -th application of $\mathcal{F}_0$ is computed. In Game (4, $j - 1$) we check

whether the input rk_{j-1} occurred before in $\mathcal{M}_{\mathcal{F}_0}$, while we do not perform this check in Game $(4, j)$. Observing that all $\text{prf}_{\text{key}_i}$ for $i \leq j-1$ have been chosen uniformly at random from $\{0, 1\}^\lambda$, the probability that $\text{mac}_{\text{key}_i} \mapsto z$, for some z , occurs in $\mathcal{M}_{\mathcal{F}}$ is upper bounded by $\frac{j-1}{2^\lambda}$. Thus, in total the probability that games Game $(4, 0)$ and Game $(4, Q)$ differ is upper bounded by $\frac{Q^2}{2^{2\lambda}}$, which is negligible. Finally, note that Game $(4, Q)$ and Game 5 are identical.

Game 5 and Game 6 are computationally indistinguishable, as a distinguisher for them could be turned into one for the pseudorandomness of PRF. Then the distinguishability from Game 6 to Game 9 follows in the same way as that from Game 2 to Game 5, except that the total probability that games Game $(8, 0)$ and Game $(8, Q)$ differ is upper bounded by $\frac{Q^2}{2^{2\lambda}}$, which is negligible as well.

Game 9 and Game 10 are computationally indistinguishable, as a distinguisher for them could be turned into one for the security of KDF. Then the distinguishability from Game 10 to Game 13 follows in the same way as that from Game 6 to Game 9, where the total probability that games Game $(12, 0)$ and Game $(12, Q)$ differ is upper bounded by $\frac{Q^2}{2^{2\lambda}}$, which is negligible.

Game 13 and Game 14 are computationally indistinguishable, as a distinguisher for them could be turned into one for the security of KDF. Then the distinguishability from Game 14 to Game 17 follows in the same way as that from Game 10 to Game 13, except that the total probability that games Game $(16, 0)$ and Game $(16, Q)$ differ is upper bounded by $\frac{Q^2}{2^{2\lambda}}$, which is negligible as well.

Finally, note that Game $(16, Q)$ and Game 17 are identical. $\square$

3. IND-CCA Security for Covert Messaging in Symmetric Ratchet

Definition 2 (IND-CCA for Covert Channel in Symmetric Ratchet). Let $\mathcal{A}$ be a PPT algorithm. The security experiment for indistinguishability under adaptive chosen ciphertext attack is $\text{Exp}_{\text{MAC}}^{\text{ind-cca}}$ shown in Figure 7. The advantage of $\mathcal{A}$ in the experiment is

$$\text{Adv}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr[\text{Exp}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A}) \rightarrow 1] - \frac{1}{2} \right|$$

Theorem 4. *If the Decisional Diffie–Hellman problem is hard in $\mathbb{G}$, MAC used in Algorithm 9 is modelled as a secure PRF and all KDFs involved are secure KDFs, for every PPT adversary $\mathcal{A}$, it holds that $\text{Adv}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{negl}(\lambda)$.*

Proof. Let $\Pr[\text{Game } i]$ be the adversary's success probability in game Game i , Q be the total number of oracle queries.

Game 0: This game is exactly the IND-CCA experiment and oracle. Hence,

$$\text{Adv}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr[\text{Game } 0] - \frac{1}{2} \right|$$

$\text{Exp}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A})$
1 : $rk_0 \leftarrow \{0, 1\}^\lambda$ 2 : $(sk_A, pk_A) \leftarrow \text{Keygen}(\lambda)$ 3 : $(sk_B, pk_B) \leftarrow \text{Keygen}(\lambda)$ 4 : $dh_{\text{out}} \leftarrow \text{DH}(sk_A, pk_B)$ 5 : $\text{info} = dh_{\text{out}} \text{ or } rk$ 6 : $S = \emptyset$ 7 : $b \leftarrow \$\{0, 1\}$ 8 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{fMAC}_b}(\cdot, \cdot, \cdot), \mathcal{O}_{\text{aDec-MAC}}(\cdot)}(1^\lambda, 1^\ell, \text{info})$ 9 : if $b' = b$ then 10 : return 1 11 : else 12 : return 0 $\mathcal{O}_{\text{fMAC}_b}(\overrightarrow{\text{ams}}_0, \overrightarrow{\text{ams}}_1, \overrightarrow{m})$
1 : for $i \in [0, \ell - 1]$ do 2 : $(rk_{i+1}, ck_i) \leftarrow \text{KDF}_{rk}(rk_i, dh_{\text{out}})$ 3 : $(ck_{i+1}, mk_i) \leftarrow \text{KDF}_{ck}(ck_i)$ 4 : $(\text{aes}_{\text{key}_{i+1}}, \text{mac}_{\text{key}_{i+1}}, \text{IV}_{i+1}) \leftarrow \text{KDF}_{mk}(mk_i)$ 5 : $ct_{i+1} \leftarrow \text{AES}_{\text{CBC}}(\text{aes}_{\text{key}_{i+1}}, \text{IV}_{i+1}, m_{i+1})$ 6 : $C \leftarrow \text{fakeMAC}(\overrightarrow{\text{mac}}_{\text{key}}, \overrightarrow{ct}, \overrightarrow{ad}, \overrightarrow{\text{ams}}_b)$ 7 : $S = S \cup \{C\}$ 8 : return C $\mathcal{O}_{\text{aDec-MAC}}(C^*)$
1 : if $C^* \in S$ 2 : return $\perp$ 3 : else 4 : return $\text{aDec}(\overrightarrow{\text{mac}}_{\text{key}}, \overrightarrow{ct}, \overrightarrow{ad}, C^*)$

Figure 7: Security experiment for IND-CCA for Covert Channel in Symmetric Ratchet

Game 1: In this game, we replace PRF in the decryption algorithm aDec with a random function $\mathcal{F} : \{0, 1\}^* \mapsto \{0, 1\}^\lambda$.

Let $\mathcal{B}_1$ be a PPT adversary against PRF security. By the security of PRF, we have

$$|\Pr[\text{Game } 0] - \Pr[\text{Game } 1]| \leq \text{Adv}^{\text{prf}}(\mathcal{B}_1)$$

Game 2: In this game, we define $\mathcal{F}$ iteratively by lazy sampling: we maintain a table for $\mathcal{F}$, and on input T : if T is already in the table, return the stored σ , otherwise sample $\sigma \leftarrow \$\{0, 1\}^\lambda$ uniformly, store and return it. Game 1 and Game 2 use equivalent methods to define $\mathcal{F}$, and thus

$$\Pr[\text{Game } 1] = \Pr[\text{Game } 2]$$

Game 3: We first define a Forge event: the aDec algorithm accepts a *fresh* ciphertext. In Game 2, for any fresh T^* , $\sigma = \text{PRF}(\text{mac}_{\text{key}}, T^*)$ is uniform and hidden. Acceptance requires guessing $\sigma = \text{mac}_{\ell+1}$. As MAC space is of λ bits, this succeeds with probability $\frac{1}{2^\lambda}$ per query, and thus

$$\Pr[\text{Forge}] \leq \frac{Q}{2^\lambda}.$$

Now in this game, it is identical to Game 2 except that on the first accepting fresh decryption query, the decryption oracle directly aborts. Then,

$$|\Pr[\text{Game 2}] - \Pr[\text{Game 3}]| \leq \Pr[\text{Forge}]$$

Game 4: In this game, we replace mac_i for $i \in [1, \ell]$ with fresh random elements.

Let $\mathcal{B}_2$ be a PPT adversary against anamorphic security we defined before, then we have

$$|\Pr[\text{Game 3}] - \Pr[\text{Game 4}]| \leq \text{Adv}_{\text{MAC}}^{\text{ana}}(\mathcal{B}_2).$$

Game 5: In the final game, all mac_i have been replaced by random values, and the decryption oracle gives no information. This is identical to the previous Game 4 and the adversary's entire view does not depend on the hidden bit b . Therefore, we have

$$\Pr[\text{Game 5}] = \Pr[\text{Game 4}] = \frac{1}{2}.$$

Combining the hops, we derive

$$\text{Adv}_{\text{MAC}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{Adv}^{\text{prf}}(\mathcal{B}_1) + \text{Adv}_{\text{MAC}}^{\text{ana}}(\mathcal{B}_2) + \frac{Q}{2^\lambda}$$

which is negligible. $\square$